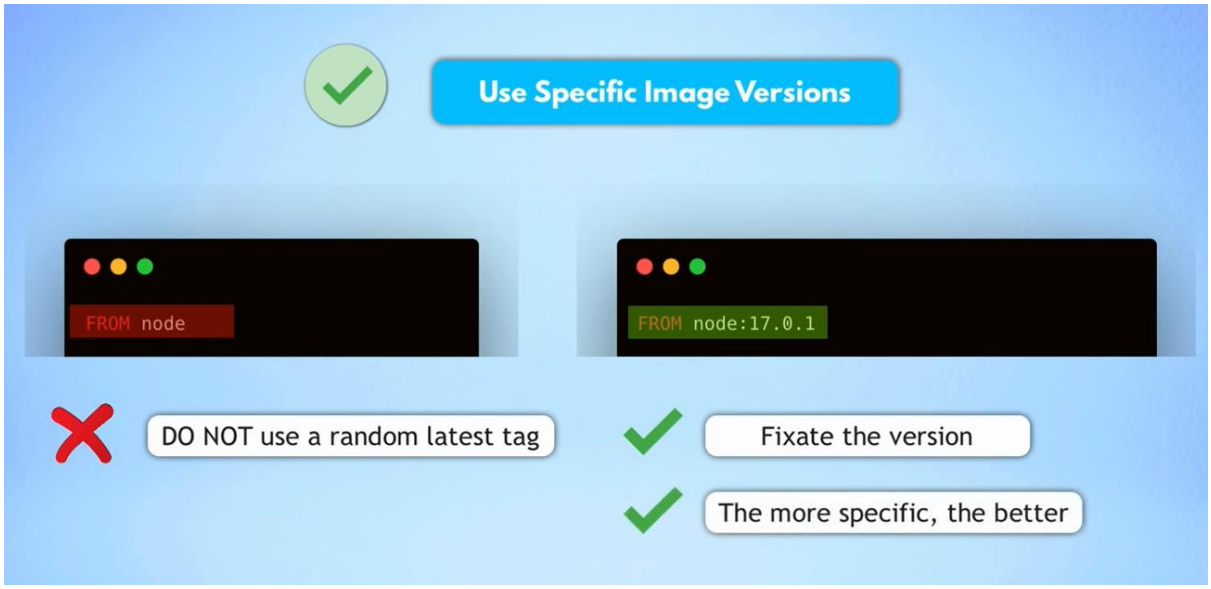
Docker Best Practices:

1. **Use Official Docker Image as Base Image**
   - Cleaner Dockerfile
   - Official Docker Image
   - Verified and already built with best practices

2. **Use specific Image tag**
   a. **It will use the latest tag**
   b. **FROM node => From node: latest**
   c. **You might get different docker image versions**
   d. **Might break stuff**
   e. **Latest tag is unpredictable**



3. **Use Small-Sized Official Images**
    a. **Smaller Images**
    b. **Less storage spaces**
    c. **Transfer Images faster**
    d. **Minimize surface attacks**
        i. **Larger Image Size**
        ii. **More security Vulnerabilities**
        iii. **Introduce unnecessary security issues from the beginning**

4. **Optimizing Caching Image Layers**
    a. **docker image history <image id>**

Optimize Caching Image Layers

Dockerfile Instructions

1) Use Node Alpine Image as Base Image
2) Go to /app folder
3) Copy project files into the Docker Image
4) Install project dependencies with "npm install"

```
FROM node:17.0.1-alpine
WORKDIR /app
COPY myapp /app
RUN npm install --production
CMD ["node", "src/index.js"]
```

Why isn't it used from cache?

- Once a layer changes, **all following layers are re-created** as well

Local Cache

Cache invalidated
Cache invalidated
Cache invalidated

```
FROM node:17.0.1-alpine      From Cache
WORKDIR /app                 From Cache
COPY myapp /app              ✗
RUN npm install --production ✗
CMD ["node", "src/index.js"] ✗
```

Optimize Caching Image Layers

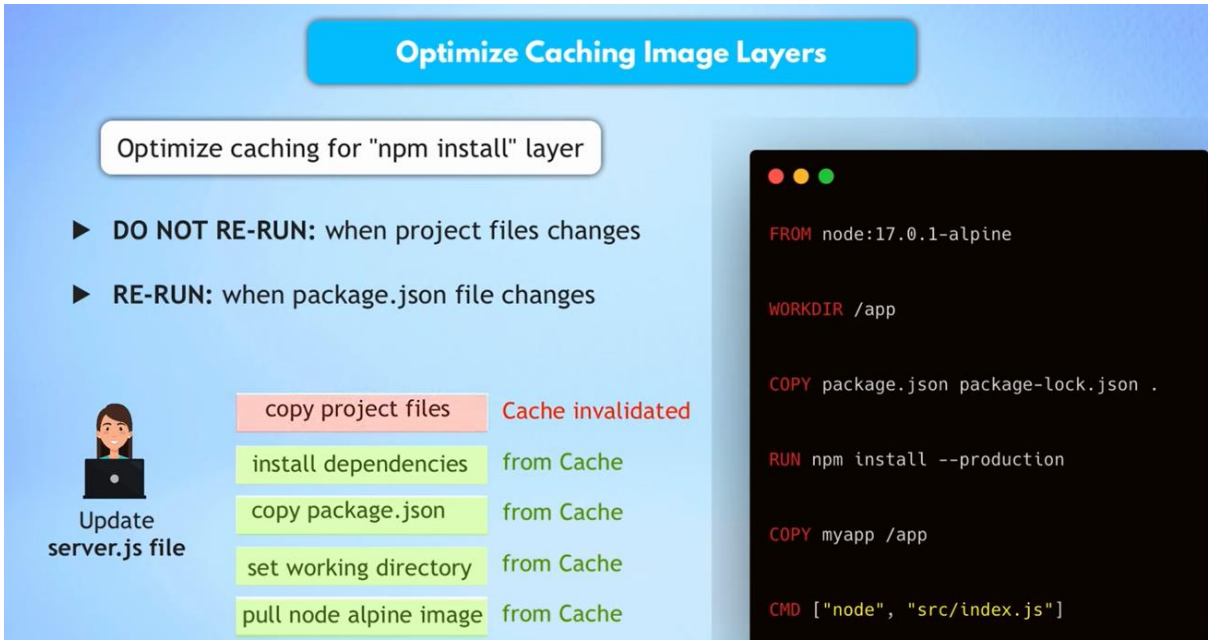
Optimize caching for "npm install" layer

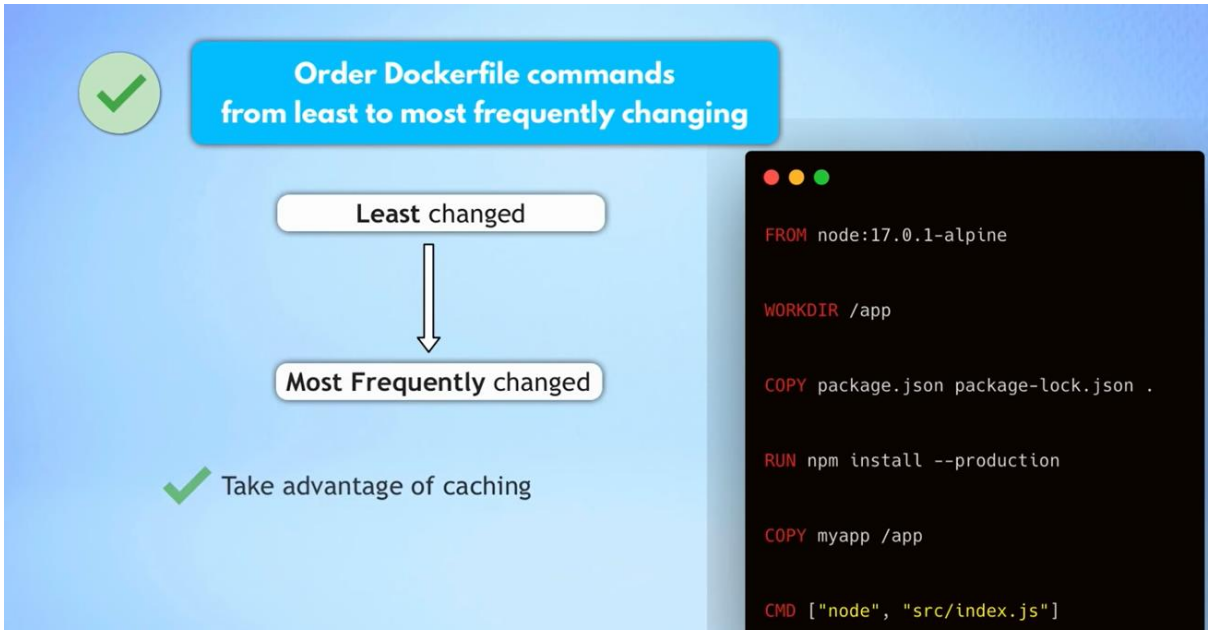
- **DO NOT RE-RUN:** when project files changes
- **RE-RUN:** when package.json file changes

copy project files
install dependencies
copy package.json
set working directory
pull node alpine image

Restructure Dockerfile instructions

```
FROM node:17.0.1-alpine
WORKDIR /app
COPY package.json package-lock.json .
RUN npm install --production
COPY myapp /app
CMD ["node", "src/index.js"]
```

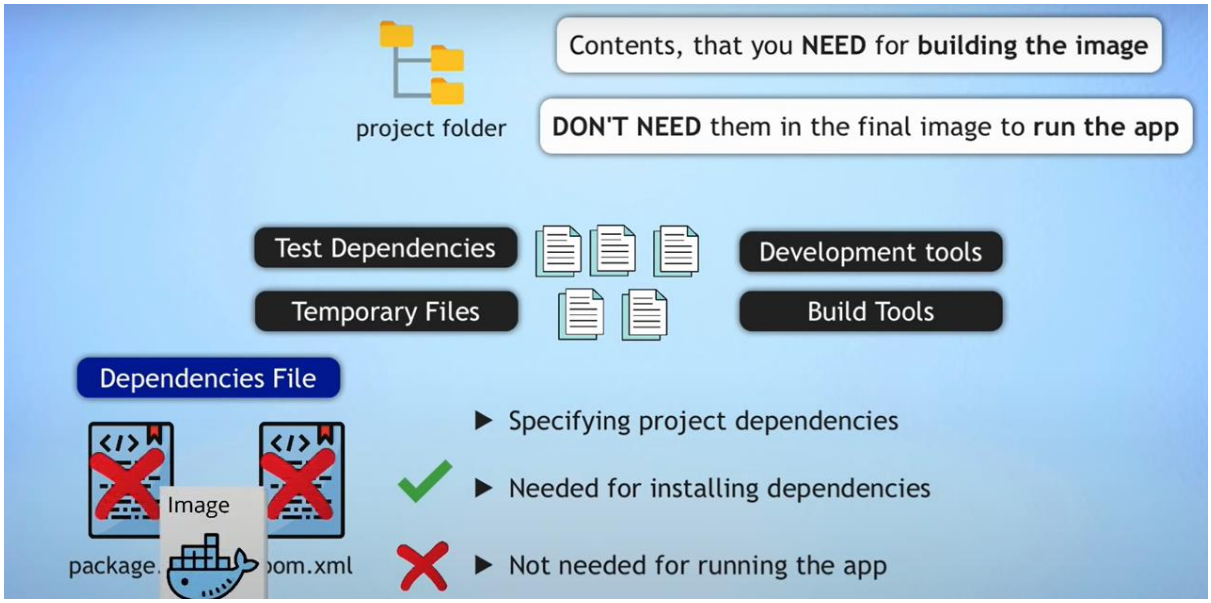


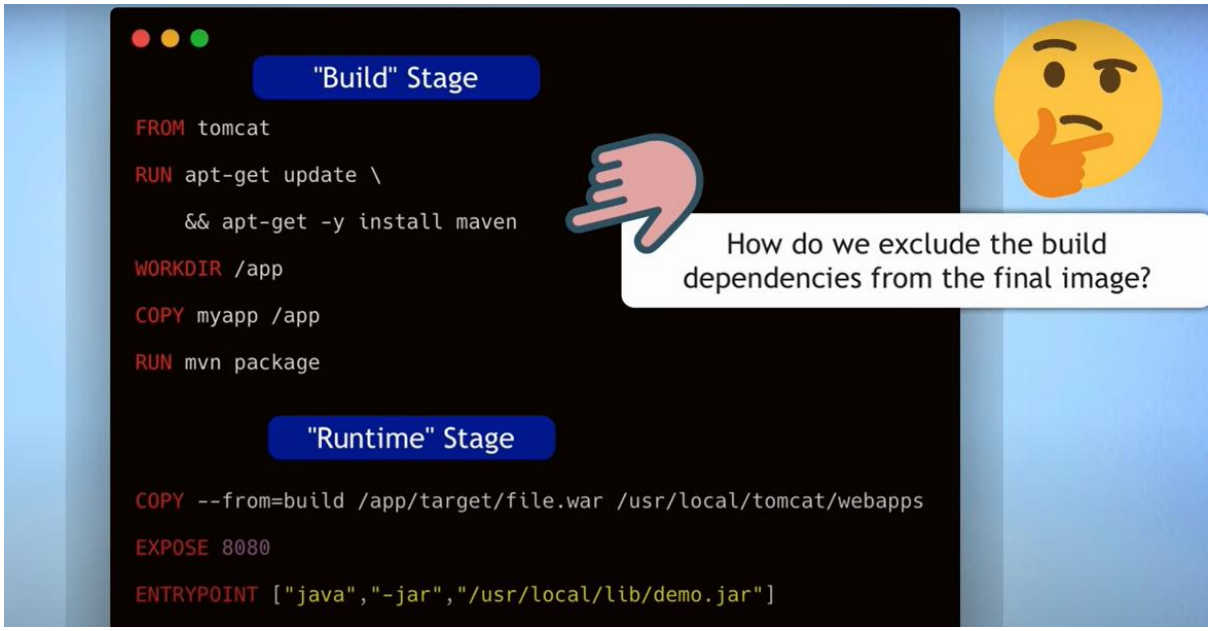


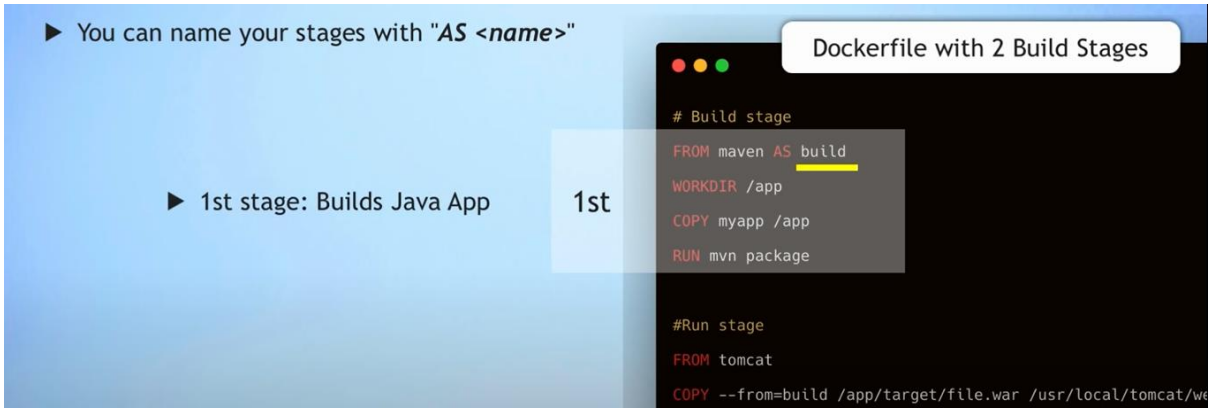
5. **Use .dockerignore to explicitly exclude files and folders**
    a. **We don't need everything inside the Docker image**
    b. **Auto-generated folders (like target, build)**
    c. **README.md**
    d. **In Order to**
        i. **Reduce Image Size**
        ii. **Prevent Unintended secrets exposure**
        iii. **Create .dockerignore file in the root directory**
        iv. **Ignore .git and .cache**
6. **Make use of "multi-Stage" build**
    a. **How do we exclude the build dependency from the final image?**

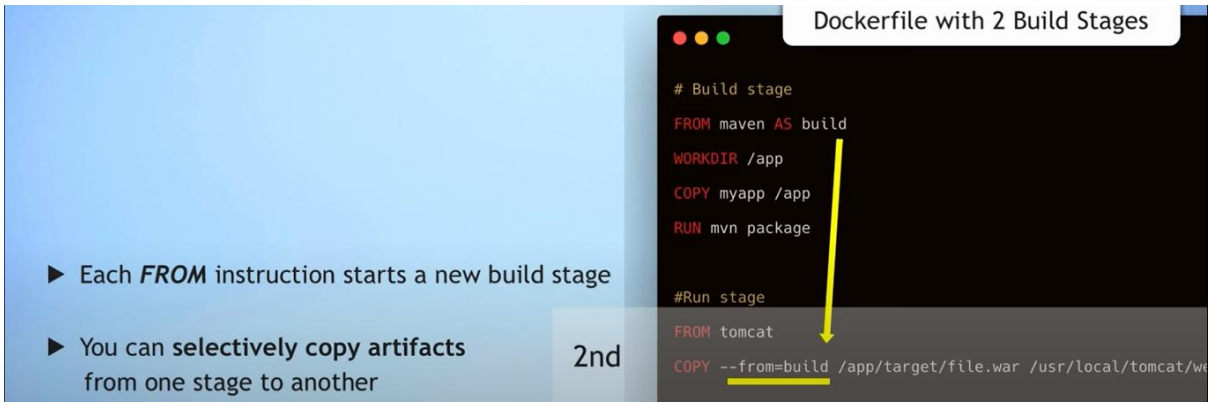
b. Build Stage/ Runtime Stage

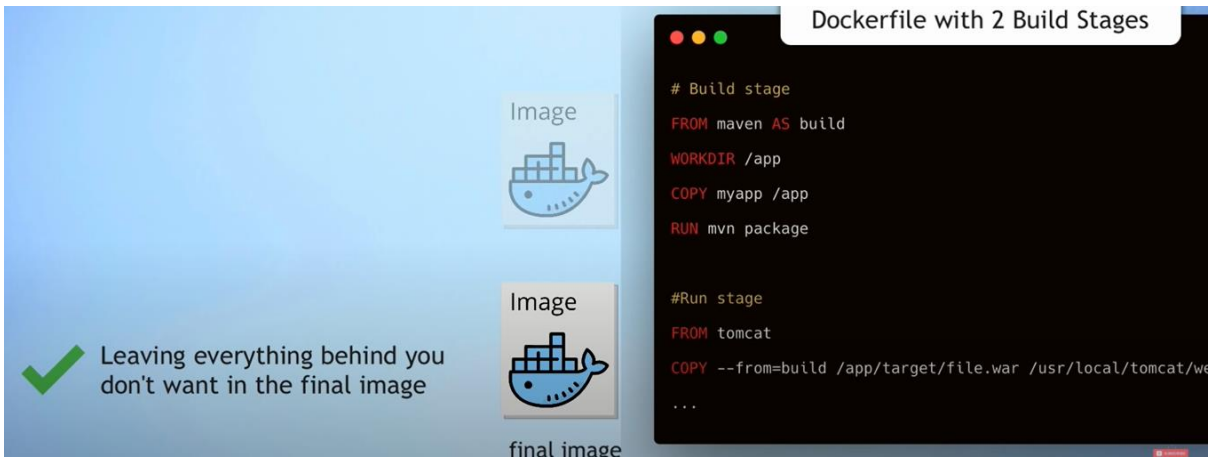
c. Contents, that you need for building the image. DON'T NEED them in the final image to run the app



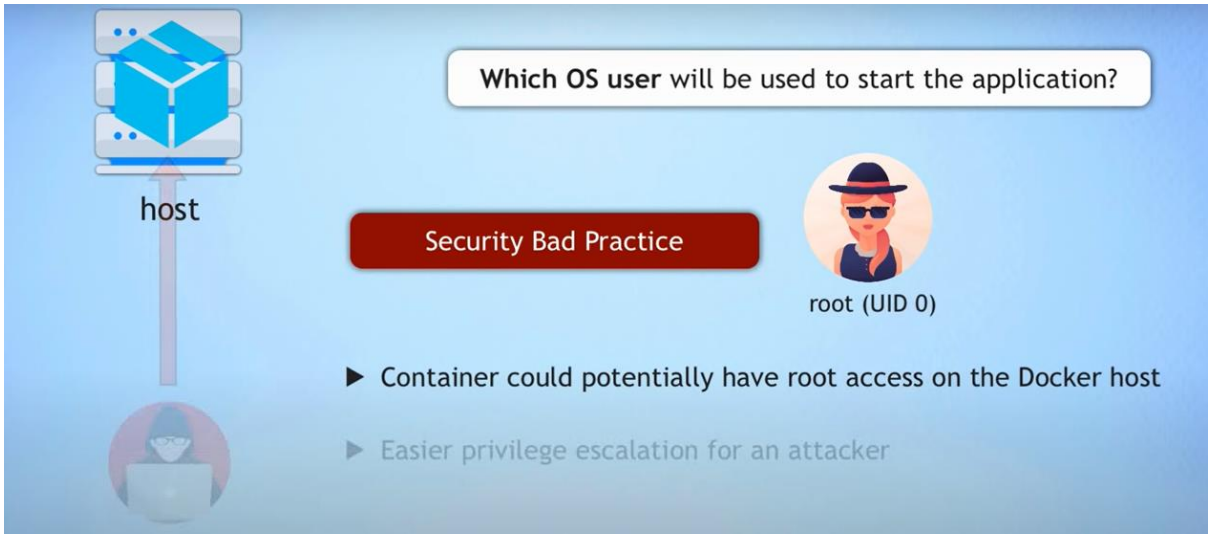


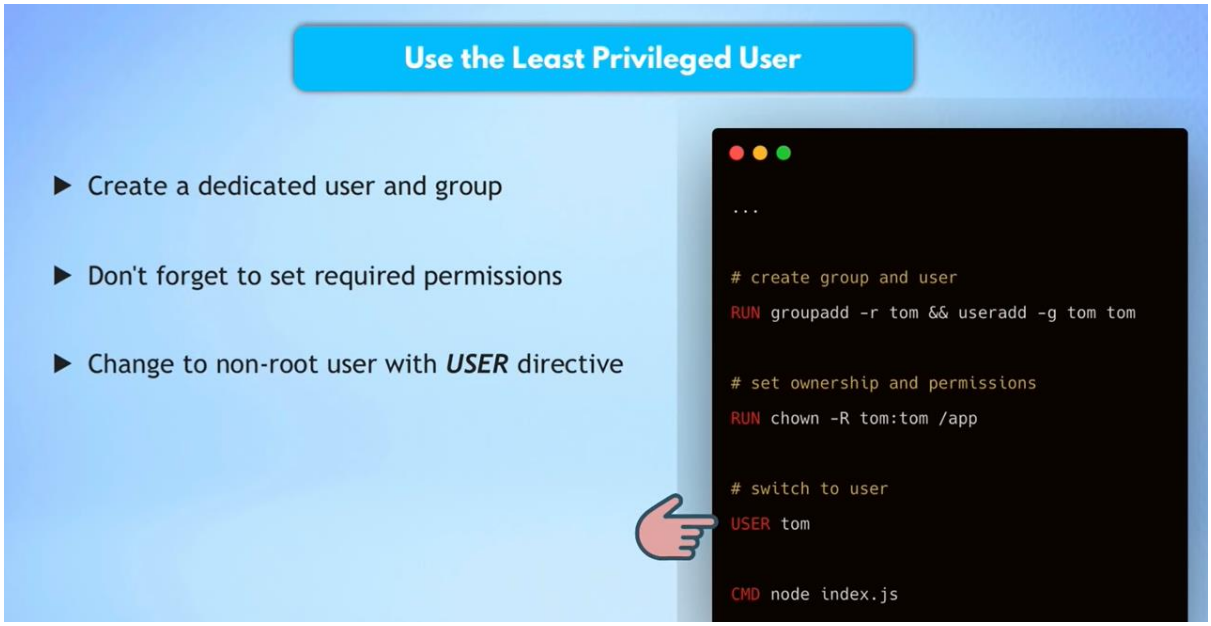

You can name your stages with "AS <name>"
1st stage: Builds Java App
1st
Dockerfile with 2 Build Stages
Build stage
FROM maven AS build
WORKDIR /app
COPY myapp /app
RUN mvn package
#Run stage
FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/we


Each FROM instruction starts a new build stage
You can selectively copy artifacts from one stage to another
2nd
Dockerfile with 2 Build Stages
Build stage
FROM maven AS build
WORKDIR /app
COPY myapp /app
RUN mvn package
#Run stage
FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/we


Leaving everything behind you don't want in the final image
Image
Image
final image
Dockerfile with 2 Build Stages
Build stage
FROM maven AS build
WORKDIR /app
COPY myapp /app
RUN mvn package
#Run stage
FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/we
...

7. **Use the Least Privileged User**





8. **Scan Your Image for Vulnerabilities using docker scan myapp:1.0**
    a. **How to validate the built Image for security vulnerabilities?**
    b. **Docker uses Snyk service for the vulnerability scan.**
    c. **You can configure DockerHub, to scan the Image automatically when the image gets pushed to the Docker repo**
    d. **Integrate in CI/CD**